

Dance4Life: Implementação de um Sistema Multiagente para a Promoção do Envelhecimento Ativo

Carlos da Mota Bergueira (pg11605@alunos.uminho.pt)
Diego Jefferson Mendes Silva (pg59999@alunos.uminho.pt)
Filipa Araújo Pereira (pg42201@alunos.uminho.pt)
Rui Manuel Martins Marques Rodrigues (pg7942@alunos.uminho.pt)

Universidade do Minho, Braga, Portugal

Abstract. O presente relatório descreve a conceção, modelação e implementação do protótipo Dance4Life, um ecossistema distribuído baseado em Sistemas Multiagente (SMA), desenvolvido em Python com a framework SPADE (Smart Python Agent Development Environment). O domínio de aplicação é o envelhecimento ativo, através da integração de dados de atividade física, contexto ambiental e emparelhamento social. A arquitetura implementada inclui um agente emissor na camada API (BaseSenderAgent) e seis agentes especializados em background (SensorAgent, CoordinatorAgent, HarAgent, EnvironmentAgent, DatabaseAgent e ClusteringAgent). A comunicação segue uma abordagem compatível com FIPA-ACL, com uso predominante das performativas request e inform, metadados de ontologia e conversation-id, e serialização de payloads com jsonpickle. Os resultados demonstram a viabilidade da abordagem multiagente para orquestração distribuída de pipelines edge-to-cloud, desde a recolha via API até à persistência em Firebase Firestore. O relatório apresenta ainda uma análise crítica das limitações observadas no código atual, nomeadamente ausência de handshake completo agree/failure, validação parcial de ontologias e acoplamento estático entre agentes, propondo melhorias para aumentar robustez, escalabilidade e conformidade com o enunciado.

Keywords: Sistemas Multiagente · SPADE · FIPA-ACL · Envelhecimento Ativo · Reconhecimento de Atividade (HAR)

1 Introdução

1.1 Domínio e Motivação

O envelhecimento demográfico global constitui um dos principais desafios sociais e de saúde pública da atualidade. A estimativa de que, até 2050, a população com mais de 65 anos atinja 1,5 mil milhões de pessoas impõe uma mudança de paradigma: a transição de cuidados clínicos hospitalares para modelos de envelhecimento ativo e monitorização remota na própria residência. É neste contexto

que surge o ecossistema concetual Dance4Life, proposto como domínio de aplicação deste trabalho prático. O Dance4Life visa promover o envelhecimento ativo através da inferência implícita de preferências do utilizador, correlacionando o movimento físico com estímulos musicais e interações sociais. A diversidade de dados envolvidos (sensores inerciais, sinais vitais, contexto sonoro e perfis sociais) não pode ser gerida eficientemente por uma arquitetura monolítica. Os SMA emergem como o paradigma tecnológico ideal para este cenário. A sua natureza modular, distribuída e assíncrona permite que entidades computacionais autónomas - os agentes - colaborem para perceber o ambiente, interpretar dados ruidosos e tomar decisões em tempo real.

1.2 Objetivos

O objetivo central deste projeto consiste na conceção e implementação de um protótipo distribuído baseado no paradigma SMA, focado no domínio do envelhecimento ativo, desenvolvido em Python com a biblioteca SPADE. Os objetivos são:

- Conceção Arquitetural: Desenhar uma arquitetura distribuída e modular, apoiada por metodologias Agent UML.
- Implementação de Agentes Inteligentes: Definir e implementar as classes de agentes atualmente operacionais no sistema (BaseSenderAgent, SensorAgent, CoordinatorAgent, HarAgent, EnvironmentAgent, DatabaseAgent e ClusteringAgent), com comportamentos CyclicBehaviour, OneShotBehaviour e PeriodicBehaviour.
- Padronização FIPA-ACL: Utilizar performativas e metadados alinhados com FIPA-ACL (performative, ontology, conversation-id), documentando o grau de conformidade efetivamente alcançado.
- Transmissão de Dados Estruturados: Garantir que o intercâmbio de mensagens utiliza payloads estruturados serializados com jsonpickle, reduzindo parsing manual e ambiguidade semântica.
- Orquestração Assíncrona: Desenvolver mecanismos de coordenação no CoordinatorAgent para gerir o pipeline sequencial sem bloqueios na thread principal.

1.3 Estrutura do Relatório

O relatório está organizado do seguinte modo: a Secção 2 apresenta a conceção e modelação do sistema (topologia, agentes e protocolos); a Secção 3 detalha a implementação técnica e o fluxo de dados; a Secção 4 apresenta os resultados e a análise crítica; e a Secção 5 conclui com recomendações de trabalho futuro.

1.4 Fundamentação Teórica

O enquadramento teórico deste trabalho assenta nos materiais pedagógicos da unidade curricular [19–22], em particular no texto de Novais e Analide [22] sobre

Agentes Inteligentes e Sistemas Multiagente, complementado pelos trabalhos fundacionais da área.

Conceito de Agente e Propriedades. Wooldridge [23] define um agente como um sistema computacional capaz de revelar uma ação autônoma e flexível num determinado universo de discurso. A flexibilidade resulta de quatro propriedades nucleares da *noção fraca* de agente [24]: **autonomia** (operar sem intervenção direta de terceiros), **reatividade** (detetar e responder a mudanças no ambiente), **pró-atividade** (tomar iniciativa guiada por objetivos) e **sociabilidade** (interagir com outros agentes de forma cooperativa ou competitiva). No Dance4Life estas propriedades têm correspondência direta: os comportamentos CyclicBehaviour detetam mensagens ACL (reatividade), cada agente processa dados de forma independente (autonomia) e o conjunto coopera para concluir o pipeline de processamento (sociabilidade).

Arquiteturas de Agentes. Novais e Analide [22] sistematizam três paradigmas arquiteturais: *deliberativo* (com representação simbólica do ambiente e planeamento de ações), *reativo* (comportamento emergente de regras condição/ação, sem representação interna) e *híbrido* (combinação de ambos, com prioridade à camada reativa para resposta rápida a estímulos externos). Esta taxonomia é aplicada a cada agente do sistema na Secção 2.2.

Sistemas Multiagente e Coordenação. Um SMA compreende um conjunto de entidades que cooperam para solucionar um problema que ultrapassa as suas capacidades individuais [26, 25]. A interação entre agentes requer três elementos fundamentais: plataforma de comunicação, linguagem de comunicação e ontologia partilhada [22]. No Dance4Life, estes requisitos são satisfeitos pelo protocolo XMPP (plataforma), pela semântica FIPA-ACL [21] (linguagem) e pelas ontologias *sensor_activity*, *movement_recommendation* e *matching*. A coordenação adotada é **cooperativa** [22]: todos os agentes partilham o objetivo de processar e persistir dados de atividade do utilizador, sem interesses conflitantes, contrastando com modelos competitivos baseados em negociação ou leilão, que seriam mais adequados a cenários multiutilizador com recursos disputados.

2 Conceção e Modelação do Sistema

A fase de conceção seguiu os princípios da metodologia de modelação baseada em agentes (Agent UML – AUML) [22]. O desenvolvimento de um ecossistema focado no envelhecimento ativo exige o processamento de dados heterogêneos em tempo real (fisiológicos, inerciais e ambientais). A arquitetura de software foi desenhada para decompor responsabilidades computacionais em agentes especializados, segundo o paradigma SMA de Weiss [25], que privilegia a distribuição funcional e a interação cooperativa entre entidades autónomas em vez de sistemas monolíticos centralizados.

2.1 Arquitetura do Sistema Multiagente (Topologia)

Na implementação atual, a arquitetura do Dance4Life adota uma topologia distribuída em cadeia, com entrada HTTP e processamento por agentes SPADE

em background. A camada de entrada é assegurada por uma API Flask, que instancia o BaseSenderAgent com identidade XMPP `api_agent@localhost`. Este agente emissor funciona como gateway lógico entre pedidos REST e o ecossistema multiagente.

O fluxo principal de atividade segue o percurso `SensorAgent` $\rightarrow$ `CoordinatorAgent` $\rightarrow$ `HarAgent` $\rightarrow$ `EnvironmentAgent` $\rightarrow$ `DatabaseAgent`. Em paralelo, existe um ramo de emparelhamento social (matching), iniciado na API e delegado ao `ClusteringAgent`, que processa convites e encaminha dados para persistência através do `CoordinatorAgent` e do `DatabaseAgent`.

O `CoordinatorAgent` mantém o papel de nó de coordenação no encadeamento principal, mas sem impor handshake completo de confirmação em cada salto. Esta opção simplifica o protótipo e permite demonstrar a integração ponta-a-ponta com Firebase; contudo, introduz limitações de robustez que são discutidas na análise crítica.

A camada de processamento especializado é composta por quatro agentes distintos:

- `HarAgent`, responsável pela passagem da fase de interpretação de atividade;
- `EnvironmentAgent`, que enriquece os dados com temperatura obtida por sensor externo;
- `ClusteringAgent`, que aplica lógica de agrupamento/convite social para a ontologia matching;
- `DatabaseAgent`, que assegura a persistência de atividades, recomendações e matching em Firebase Firestore.

Todos os agentes comunicam por mensagens ACL sobre XMPP, com payloads serializados em jsonpickle e metadados de ontologia e conversation-id. Esta organização mantém o sistema modular e facilita a substituição incremental de componentes, por exemplo na evolução futura do `HarAgent` para modelos de inferência reais.

Table 1. Topologia e comunicações do sistema Dance4Life.

Agente	Tipo / Papel	Communicates With
BaseSenderAgent (API)	Gateway REST para ACL	SensorAgent, ClusteringAgent
SensorAgent	Reativo de entrada	BaseSenderAgent, CoordinatorAgent
CoordinatorAgent	Orquestrador de encaminhamento	SensorAgent, HarAgent, DatabaseAgent
HarAgent	Encaminhamento da fase HAR	CoordinatorAgent, EnvironmentAgent
EnvironmentAgent	Enriquecimento ambiental	HarAgent, DatabaseAgent
ClusteringAgent	Processamento de matching social	BaseSenderAgent, CoordinatorAgent
DatabaseAgent	Persistência Firestore	EnvironmentAgent, CoordinatorAgent

****DIAGRAMA****

2.2 Definição dos Agentes

Classes, Behaviours e Performativas A implementação atual assenta em duas classes base: `BaseBackgroundAgent` (agentes residentes com ciclo próprio) e `BaseSenderAgent` (agente emissor associado à API). A primeira integra registo, heartbeat periódico e remoção no servidor Flask através de `RegisterBehaviour`, `HeartbeatBehaviour` e `UnregisterBehaviour`; a segunda encapsula envios pontuais com `SendMessageBehaviour`.

BaseSenderAgent (API) — Gateway Externo O `BaseSenderAgent` é iniciado no arranque da API e envia mensagens ACL para o `SensorAgent` (atividade e recomendações) e para o `ClusteringAgent` (matching). Em cada envio, define performative, ontology e conversation-id, e serializa o payload com `jsonpickle`.

SensorAgent — Entrada da Camada Física/Lógica Implementa `ReceiveSensorDataBehaviour` (`CyclicBehaviour`), recebendo pedidos da API e reencaminhando-os para o `CoordinatorAgent` com preservação do conversation-id. Suporta as ontologias `sensor_activity` e `movement_recommendation`. Seguindo a taxonomia de Novais e Analide [22], classifica-se como **reativo**: o comportamento emerge de regras condição/ação diretas (receber mensagem → encaminhar), sem representação simbólica interna do ambiente.

CoordinatorAgent — Coordenação de Encaminhamento Implementa `ReceiveCoordinatorDataBehaviour` (`CyclicBehaviour`). Para `sensor_activity`, encaminha para `HarAgent`; para `movement_recommendation` e matching, encaminha para `DatabaseAgent` após enriquecimento mínimo do payload (flag recommendation). O papel é de orquestração de encaminhamento, não de processamento analítico complexo. A lógica de decisão por ontologia e performativa introduz um elemento deliberativo, classificando este agente como **híbrido** [22]: o ciclo reativo é sobreposto por regras de roteamento baseadas no tipo de mensagem recebida, permitindo adaptação do fluxo em função do domínio.

HarAgent — Etapa HAR da Cadeia Implementa `ReceiveHarDataBehaviour` (`CyclicBehaviour`) e reencaminha pedidos de atividade para o `EnvironmentAgent`. Nesta versão do protótipo, não executa ainda inferência HAR baseada em modelo de Machine Learning; funciona como etapa modular reservada para essa integração futura. O agente é atualmente **reativo**; a futura integração de modelos CNN/LSTM transformá-lo-á numa arquitetura **deliberativa** [23], com raciocínio sobre sinais inerciais para inferência de atividade física, correspondendo à visão de Wooldridge de agentes cognitivos orientados a objetivos.

EnvironmentAgent — Enriquecimento Contextual Implementa `EnvironmentDataBehaviour` (`CyclicBehaviour`), decodifica o payload, obtém temperatura por coordenadas geográficas e adiciona o atributo `weather_temperature`

antes de encaminhar para o DatabaseAgent. A consulta à API meteorológica externa constitui uma decisão baseada no conteúdo do payload, conferindo a este agente um carácter **híbrido** [22]: o ciclo reativo é enriquecido com uma fase de aquisição de conhecimento contextual, na qual o agente percebe o ambiente externo (temperatura/humidade) antes de encaminhar para a persistência.

ClusteringAgent — Matching Social Implementa ReceiveClusteringDataBehaviour (CyclicBehaviour), processa pedidos da ontologia matching através de UserActivityClusterModel e publica convites para utilizadores via endpoint API (/set_user_match/<user_id>). Posteriormente, encaminha o resultado consolidado para o CoordinatorAgent. Este é, dos agentes implementados, o mais próximo de uma arquitetura **deliberativa** [23]: a execução de UserActivityClusterModel implica raciocínio sobre perfis de utilizadores e dados de localização para produzir decisões de emparelhamento social, aproximando-se da noção forte de agente com intencionalidade e competência [22].

DatabaseAgent — Persistência Implementa ReceiveDatabaseDataBehaviour (CyclicBehaviour), removendo visited_agents antes de persistir. Suporta três operações: save_activity, save_movement_recommendation e save_matching em Firestore, conforme a ontologia recebida. Classifica-se como **agente de serviço reativo** na tipologia de Novais e Analide [22]: executa tarefas de persistência repetitivas em resposta a pedidos externos, sem representação interna do domínio aplicacional, correspondendo ao padrão de agente estacionário dedicado a operações de I/O.

Table 2. Síntese dos agentes, behaviours, tipos e classificação arquitetural.

Agente	Behaviour	Tipo	Arquitetura	Ontologia
BaseSenderAgent	SendMessageBehaviour	OneShotBehaviour	Reativo	s_a, m_r, m
SensorAgent	ReceiveSensorDataBehaviour	CyclicBehaviour	Reativo	s_a, m_r
CoordinatorAgent	ReceiveCoordinatorDataBehaviour	CyclicBehaviour	Híbrido	s_a, m_r, m
HarAgent	ReceiveHarDataBehaviour	CyclicBehaviour	Reativo [†]	s_a
EnvironmentAgent	EnvironmentDataBehaviour	CyclicBehaviour	Híbrido	s_a
ClusteringAgent	ReceiveClusteringDataBehaviour	CyclicBehaviour	Deliberativo	m
DatabaseAgent	ReceiveDatabaseDataBehaviour	CyclicBehaviour	Reativo	s_a, m_r, m

s_a = sensor_activity; m_r = movement_recommendation; m = matching

[†] Futuro: Deliberativo após integração de modelo HAR

2.3 Protocolos de Comunicação e Performativas FIPA-ACL

A comunicação entre agentes no Dance4Life adota uma semântica compatível com FIPA-ACL, suportada pelas capacidades nativas do SPADE. Todas as mensagens relevantes incluem metadados performative, ontology e conversation-id, permitindo rastreabilidade e separação lógica de fluxos.

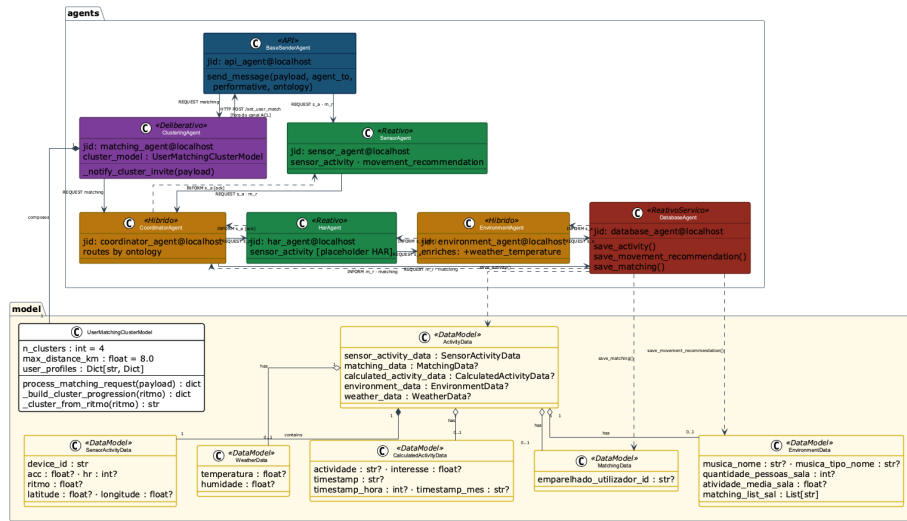


Fig.1. Diagrama de classes AAML do sistema Dance4Life. Pacotes representados: **spade** [external] (framework base), **agents.base** (agentes base), **agents** (agentes operacionais), **behaviours.lifecycle** e **behaviours.operational** (behaviours SPADE), **model** (modelos Pydantic), **services** (persistência Firestore) e **FIPA-ACL** (protocolo de comunicação). Relações: herança (◁), composição (●—), agregação (○—) e dependência (dashed).

Na versão implementada, as performativas efetivamente usadas na cadeia operacional são request e inform. As constantes agree e failure estão definidas na configuração, mas não se encontram aplicadas de forma sistemática no caminho principal. Assim, o sistema cumpre parcialmente o padrão FIPA Request Interaction Protocol: existe iniciação (request) e resolução (inform), mas não há handshake explícito em todas as interações.

Modelo de Interação O fluxo de interação principal observa a seguinte sequência:

1. Iniciação (request): o emissor envia mensagem para o próximo agente da cadeia, incluindo:
 - ontologia (sensor_activity, movement_recommendation ou matching);
 - conversation-id automático (gerado em SendMessageBehaviour);
 - payload serializado com jsonpickle.
2. Processamento e reencaminhamento: cada agente decodifica, enriquece ou transforma o payload e encaminha para o agente seguinte, mantendo os metadados essenciais.
3. Resolução (inform): após concluir a operação local (encaminhamento ou persistência), o agente responde ao emissor com performativa inform.
 - em caso de sucesso, informa o remetente imediato;
 - em caso de erro, o tratamento atual é sobretudo por exceção e logging local, sem protocolo failure padronizado entre todos os pares.

Gestão de Conversações e Isolamento de Sessões Cada mensagem criada no BaseSenderAgent inclui um conversation-id único (UUID). O identificador é propagado entre agentes durante os forwards, permitindo rastrear cada cadeia de processamento nos logs e nas mensagens.

Importa, contudo, distinguir entre propagação e isolamento completo: apesar de o metadado existir e ser preservado, o código atual não implementa ainda uma filtragem estrita por conversation-id em todos os pontos de receção. Por essa razão, o mecanismo presente garante rastreabilidade, mas a gestão de concorrência pode ser fortalecida em evolução futura.

Mesmo nesta forma, a abordagem já contribui para:

- observabilidade do fluxo ponta-a-ponta;
- depuração de interações em cadeias longas;
- preparação para isolamento rigoroso em cenários com maior concorrência.

Troca de Dados Estruturados Embora o transporte XMPP opere sobre texto, o protótipo evita strings ad hoc e adota payloads estruturados serializados com jsonpickle:

- os dados são construídos em dicionários estruturados e enriquecidos por cada etapa;

- `jsonpickle.encode/jsonpickle.decode` asseguram serialização e desserialização uniformes;
- existe um modelo de domínio em Pydantic (`model/data_model.py`) para normalização semântica, embora o pipeline corrente opere maioritariamente sobre dicionários.

Esta solução cumpre o requisito do enunciado de privilegiar objetos estruturados em detrimento de mensagens textuais sem esquema.

Table 3. Performativas FIPA-ACL utilizadas no sistema.

Performativa	Direção	Significado	Quando enviada
request	Emissor → Recetor	Pedido de execução/encaminhamento	Fluxos de atividade, matching e recomendação
inform	Recetor → Emissor	Confirmação local de processamento	Após encaminhamento/persistência
agree	Recetor → Emissor	Aceitação explícita do pedido	Definida em configuração (não aplicada de forma
failure	Recetor → Emissor	Resposta semântica de falha	Definida em configuração (não aplicada de forma

A implementação atual demonstra alinhamento prático com os princípios FIPA-ACL, mas também evidencia espaço para maturação protocolar, sobretudo na adoção sistemática de `agree/failure` e em validação semântica mais estrita.

2.4 Diagramas de Atividades AUML por Protocolo

Os três diagramas seguintes representam o comportamento do sistema segundo a notação *Agent UML* (AUML) [22], adoptando pistas de actividade (*swimlanes*) por agente e setas diferenciadas para as performativas REQUEST (encaminhamento) e INFORM (confirmação). Os diagramas foram gerados a partir do código-fonte verificado e reflectem a implementação efectiva, não um modelo idealizado.

2.5 Diagramas de Colaboração AUML por Protocolo

Os diagramas de colaboração AUML (também designados *communication diagrams* em UML 2.x) representam os agentes como instâncias nomeadas (no formato `jid:Classe`) e as ligações entre eles como canais de comunicação persistentes, com mensagens ACL numeradas sequencialmente sobre essas ligações [22]. Distinguem-se dos diagramas de actividades por enfatizarem a **estrutura da rede de comunicação** e a **ordem das trocas de mensagens**, em vez do fluxo de controlo interno de cada agente. As setas sólidas correspondem à performativa REQUEST (encaminhamento), as setas tracejadas à performativa INFORM (confirmação de processamento) e as setas duplas indicam chamadas HTTP externas fora do canal ACL.

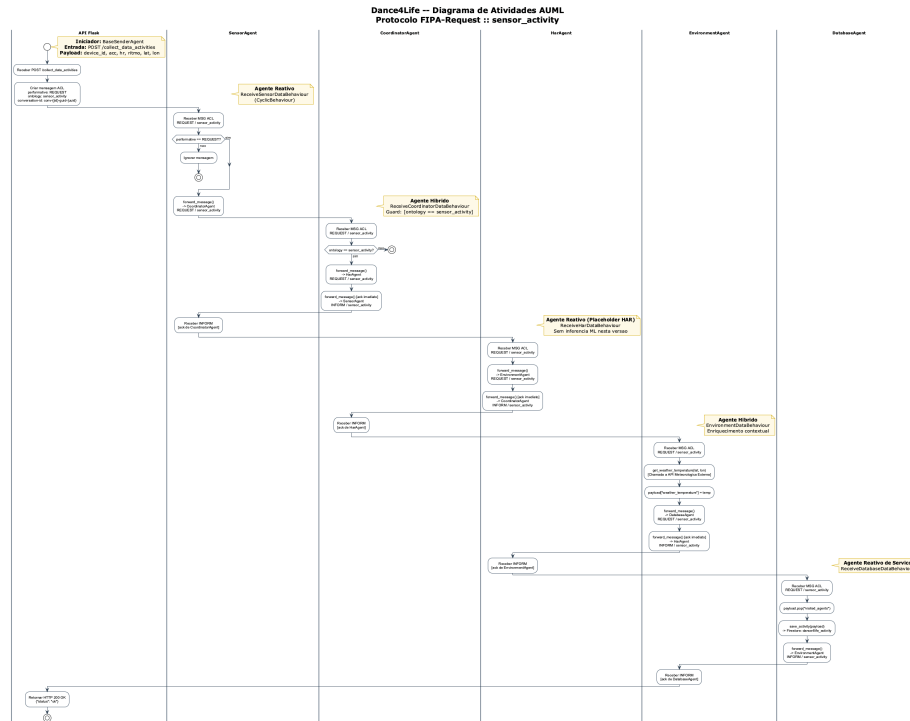


Fig.2. Diagrama de actividades AUML — Protocolo FIPA-Request, ontologia **sensor_activity**. Fluxo: API → SensorAgent → CoordinatorAgent → HarAgent → EnvironmentAgent → DatabaseAgent. Retorno INFORM na direcção inversa após cada operação.

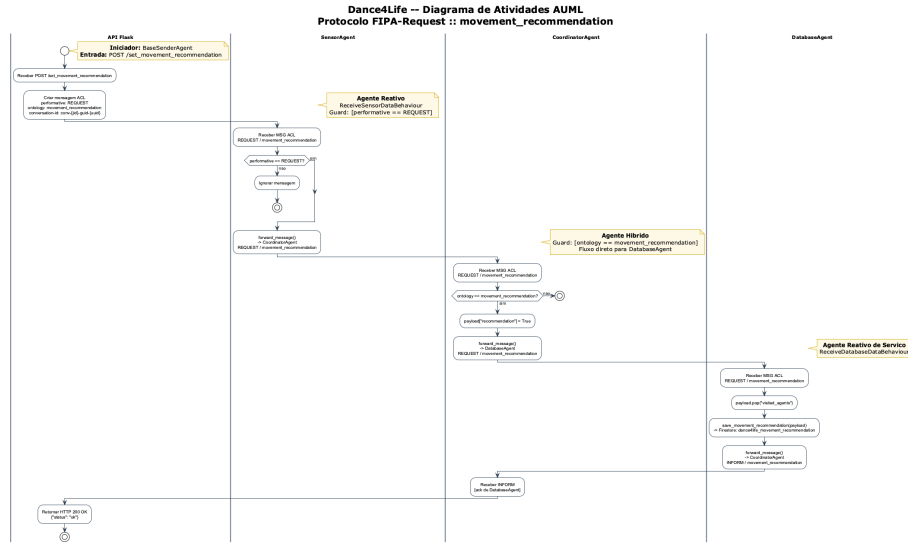


Fig. 3. Diagrama de atividades AUML — Protocolo FIPA-Request, ontologia `movement_recommendation`. Fluxo simplificado: API → SensorAgent → CoordinatorAgent → DatabaseAgent. Retorno INFORM de DatabaseAgent para CoordinatorAgent.

2.6 Diagramas de Sequência AUML por Protocolo

Os diagramas de sequência AUML representam os agentes como linhas de vida (*lifelines*) identificadas pelo par `jid:Classe<<estereótipo>>` e as mensagens ACL como setas numeradas sobre o eixo temporal (de cima para baixo) [22]. Relativamente aos diagramas de colaboração, acrescentam três elementos essenciais: os **blocos de ativação** (rectângulos sobre a lifeline, que delimitam o período de processamento de cada agente), as **chamadas internas** (auto-mensagens, que representam operações locais como enriquecimento contextual ou persistência em Firestore) e as **secções de protocolo** (bandas que separam a Iniciação FIPA-Request, o Processamento e a Resolução). As setas sólidas com ponta ($\rightarrow$) correspondem à performativa REQUEST e as setas tracejadas ($-->$) à performativa INFORM.

2.7 Statecharts: Ciclo de Vida e Pipeline de Mensagens

O diagrama 11 modela o ciclo de vida do par servidor Flask – `BaseBackgroundAgent`: arranque, registo via `POST /register_agent`, monitorização periódica por heartbeat (period = 10s), deteção de timeout (`HEARTBEAT_TIMEOUT = 30s`) pela *cleanup thread* do servidor, e paragem ordenada com `POST /unregister_agent`.

O diagrama 12 apresenta os estados internos de cada agente e as transições inter-agentes para a ontologia `sensor_activity`, a cadeia mais completa do sistema. Cada agente alterna entre *Aguardando* (`receive(timeout=10)`),

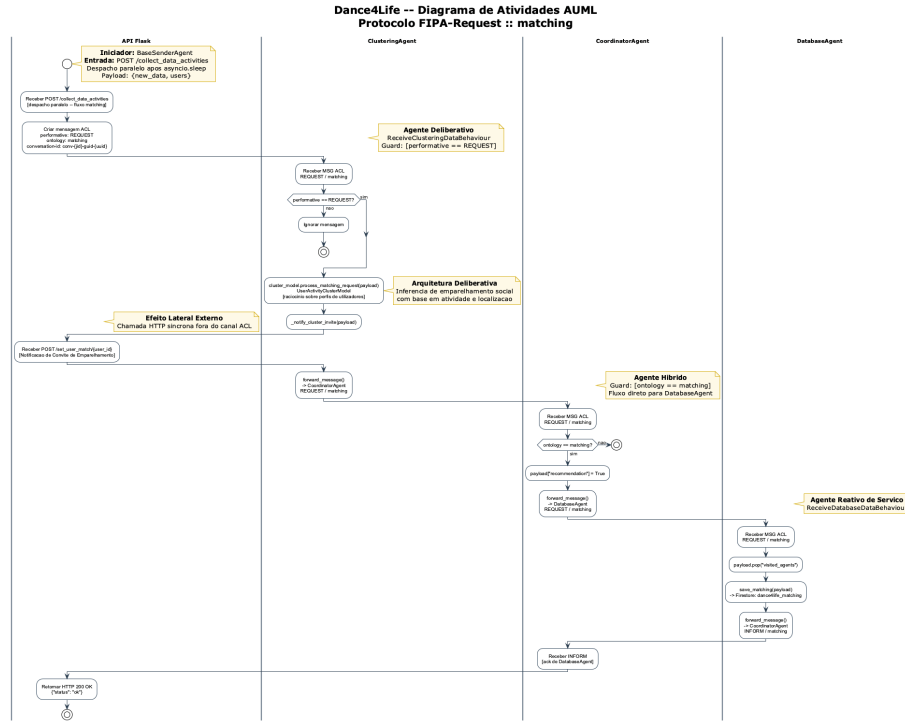


Fig.4. Diagrama de actividades AUMIL — Protocolo FIPA-Request, ontologia matching. Fluxo: API → ClusteringAgent (inferência deliberativa) → notificação HTTP lateral (/set_user_match) → CoordinatorAgent → DatabaseAgent. Retorno INFORM de DatabaseAgent para CoordinatorAgent.

Dance4Life -- Diagrama de Colaboracao AUML Protocolo FIPA-Request :: *sensor_activity*

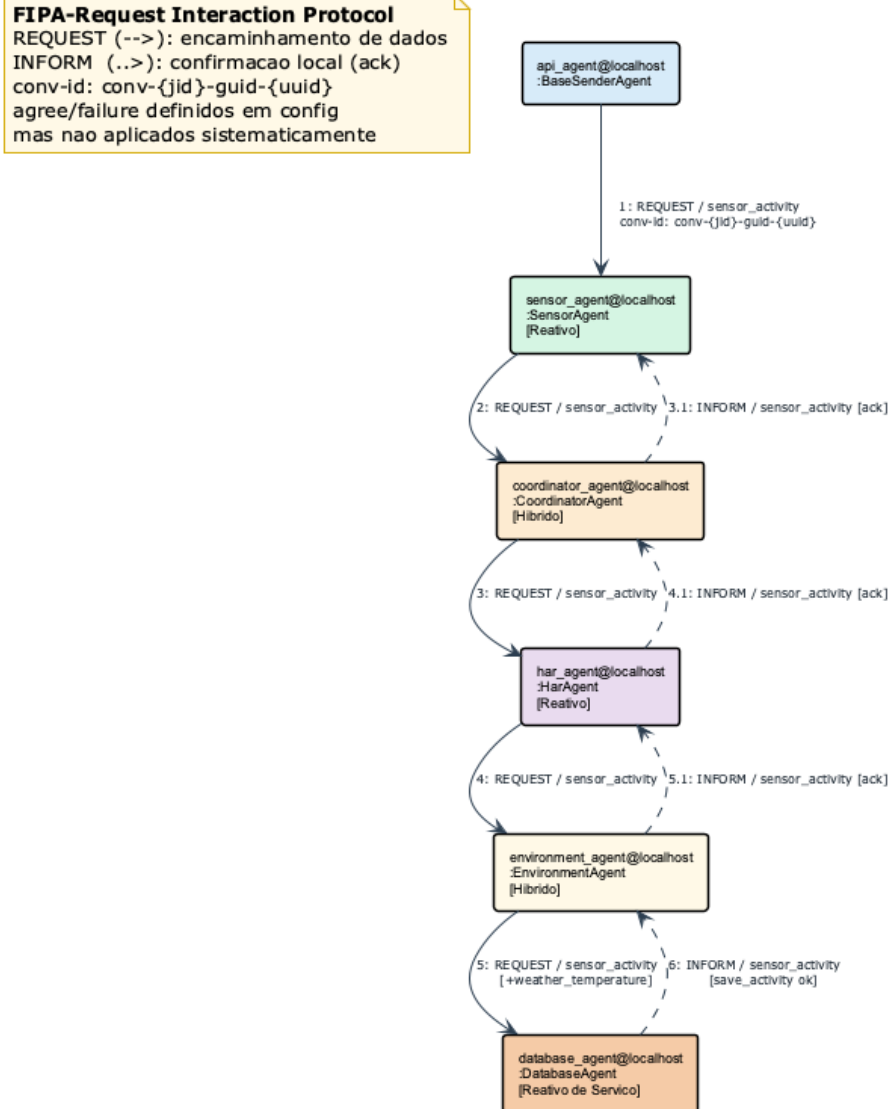


Fig. 5. Diagrama de colaboração AUML — Protocolo FIPA-Request, ontologia *sensor_activity*. Mensagens REQUEST numeradas 1–5 (setas sólidas, esquerda para a direita); mensagens INFORM 3.1, 4.1, 5.1 e 6 (setas tracejadas, direita para a esquerda). O conv-id é propagado em todas as mensagens da conversação.

Dance4Life -- Diagrama de Colaboracao AUMl Protocolo FIPA-Request :: *movement_recommendation*

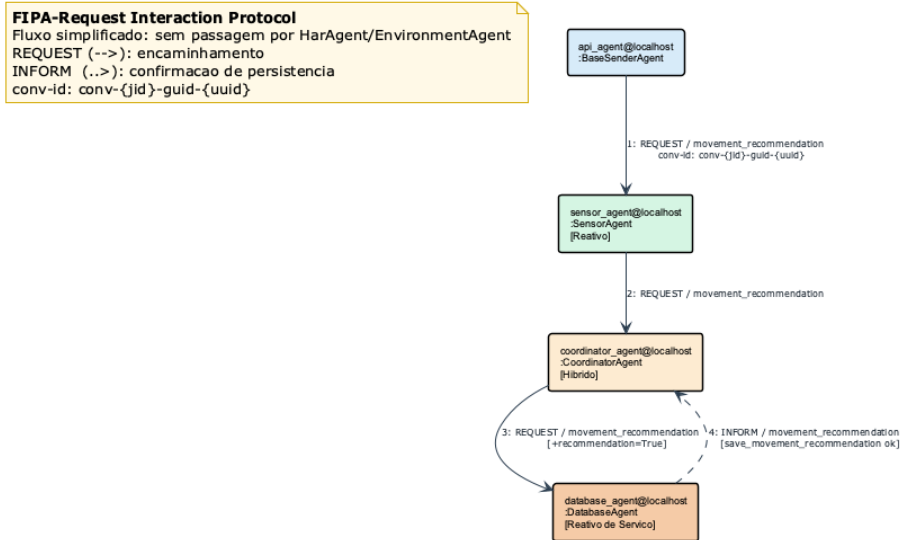


Fig. 6. Diagrama de colaboração AUMl — Protocolo FIPA-Request, ontologia *movement_recommendation*. Fluxo simplificado de quatro mensagens: REQUEST 1–3 (API → SensorAgent → CoordinatorAgent → DatabaseAgent) e INFORM 4 de confirmação de persistência.

Dance4Life -- Diagrama de Colaboracao AUML Protocolo FIPA-Request :: *matching*

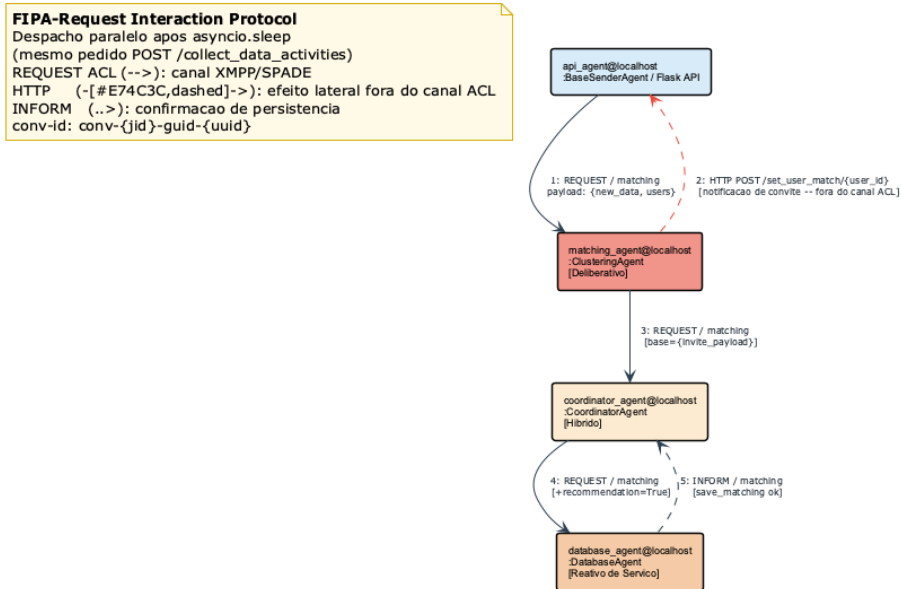


Fig. 7. Diagrama de colaboração AUML — Protocolo FIPA-Request, ontologia *matching*. A mensagem 2 representa o efeito lateral HTTP (POST /set_user_match/{id}) que o ClusteringAgent efectua directamente sobre a Flask API fora do canal ACL, antes de encaminhar o resultado via XMPP para o CoordinatorAgent (mensagem 3).

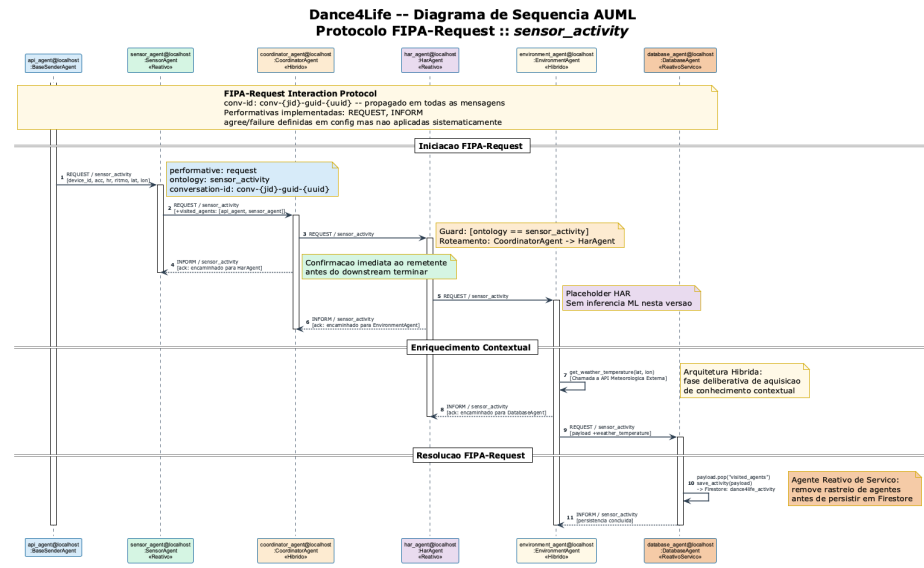


Fig.8. Diagrama de sequência AUML — Protocolo FIPA-Request, ontologia *sensor_activity*. 11 mensagens numeradas: REQUEST 1–3, 5, 9 (setas sólidas); INFORM 4, 6, 8, 11 (setas tracejadas); auto-mensagens 7 (*get_weather_temperature*) e 10 (*save_activity*). Os blocos de activação mostram o período em que cada agente aguarda resposta do downstream.

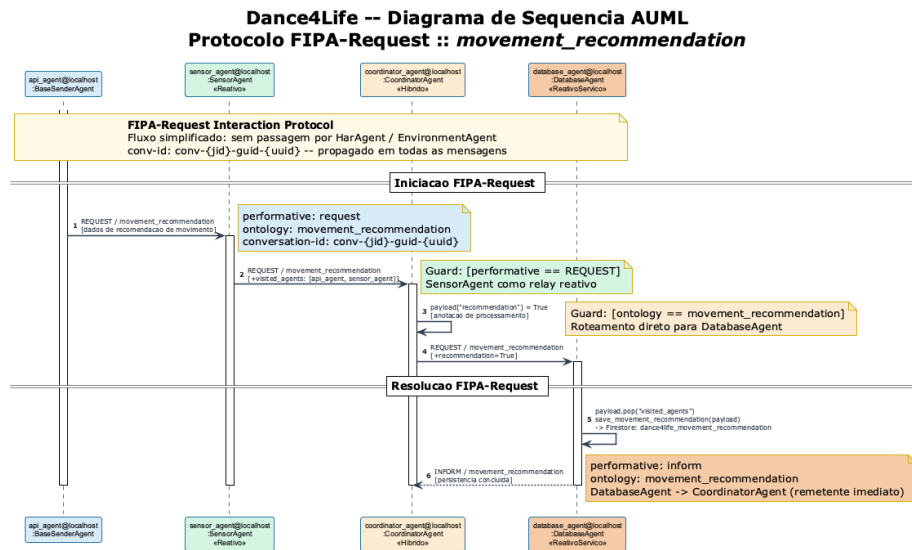


Fig. 9. Diagrama de sequência AUMI — Protocolo FIPA-Request, ontologia *movement_recommendation*. Fluxo de 6 mensagens: REQUEST 1-2 (API→SensorAgent→CoordinatorAgent), auto-mensagem 3 (payload["recommendation"]=True), REQUEST 4 (→DatabaseAgent), auto-mensagem 5 (save_movement_recommendation) e INFORM 6 de confirmação.

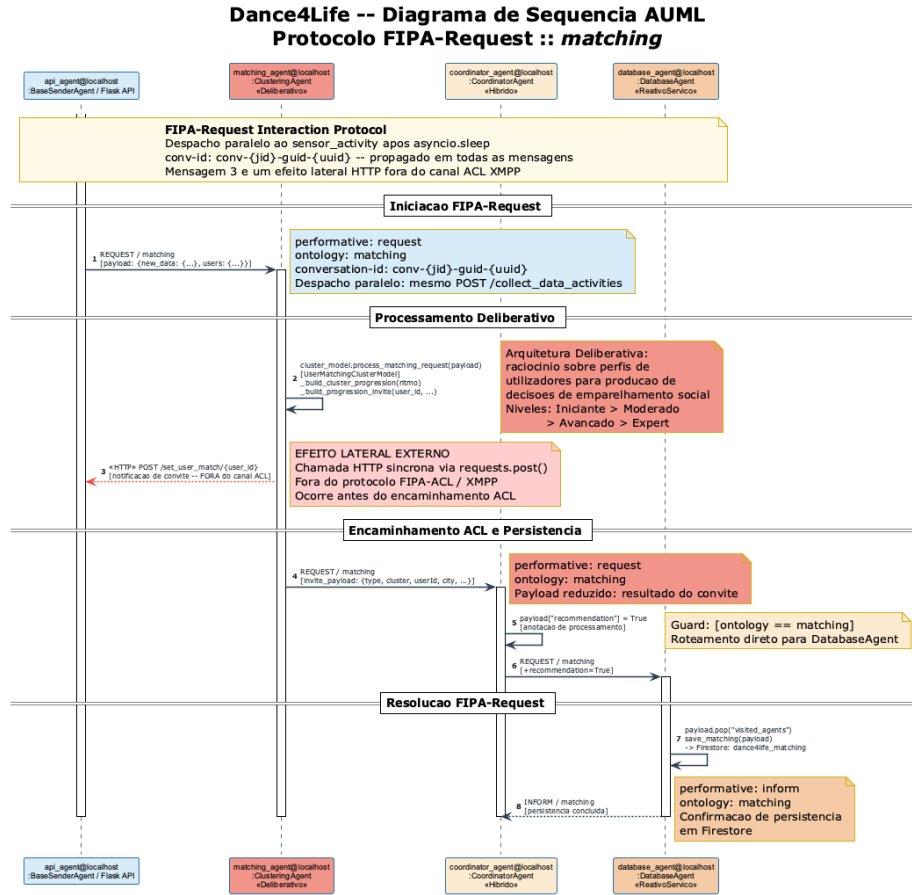


Fig.10. Diagrama de sequência AUML — Protocolo FIPA-Request, ontologia matching. A mensagem 3 (HTTP POST /set_user_match/{id}) é um efeito lateral síncrono executado pelo ClusteringAgent **fora do canal ACL** antes de retomar o protocolo FIPA-Request na mensagem 4. O processamento deliberativo do UserMatchingClusterModel é representado como auto-mensagem 2.

Processando (decodificação e verificação) e *Encaminhando* (`forward_message`), emitindo um INFORM de reconhecimento ao remetente após cada salto.

3 Implementação e Funcionamento (Baseado em SPADE)

O sistema Dance4Life foi implementado em Python e SPADE, com execução assíncrona sobre `asyncio` e comunicação ACL sobre XMPP. A solução combina uma API Flask para interação externa com uma rede de agentes residentes em loops próprios (threads dedicadas + event loop por agente), permitindo desacoplamento entre entrada HTTP e processamento multiagente.

3.1 Instalação Externa da SPADE e Ambiente de Execução

A SPADE (Smart Python Agent Development Environment) é uma framework Python para o desenvolvimento de agentes assíncronos baseados em XMPP, alinhada com os princípios de comunicação FIPA-ACL [21]. Ao contrário de plataformas como o JADE [22] (implementado em Java), o SPADE tira partido do motor assíncrono `asyncio` do Python, tornando-o particularmente adequado para ambientes de prototipagem rápida. No contexto deste projeto, a SPADE não foi integrada no stack Android; foi instalada externamente num ambiente Python autónomo localizado na pasta Python do projeto, com gestão de dependências em `pyproject.toml`.

As dependências relevantes incluem `spade`, `flask`, `jsonpickle`, `firebase-admin`, `requests` e `pandas`. A execução segue o ciclo típico:

```
python -m venv .venv
source .venv/bin/activate
pip install -e .
python main.py
```

Esta separação entre componente móvel e backend SMA simplifica a experimentação, facilita debug e permite evolução independente do subsistema multi-agente.

3.2 Fluxo de Dados — Pipeline Sequencial Assíncrono

Apesar de os SMA serem concorrentes por natureza, o protótipo implementa um pipeline encadeado. No caso `sensor_activity`, a sequência observada em código é:

- API (`BaseSenderAgent`) envia request para `SensorAgent`;
- `SensorAgent` reencaminha para `CoordinatorAgent`;
- `CoordinatorAgent` reencaminha para `HarAgent`;
- `HarAgent` reencaminha para `EnvironmentAgent`;
- `EnvironmentAgent` adiciona `weather_temperature` e envia para `DatabaseAgent`;
- `DatabaseAgent` persiste em `Firestore` e devolve inform ao emissor imediato.

Para `movement_recommendation`, o fluxo segue `API → SensorAgent → CoordinatorAgent → DatabaseAgent`. Para `matching`, a API envia para `ClusteringAgent`, que processa convites e encaminha para `CoordinatorAgent` e `DatabaseAgent`.

A assíncronia decorre da execução em behaviours cíclicos e do uso de event loops independentes por agente. O resultado é um sistema responsivo a múltiplos pedidos sem bloqueio da API principal.

****DIAGRAMA****

3.3 Gestão de Conversações e Isolamento de Mensagens

Cada envio realizado pelo `BaseSenderAgent` inclui `conversation-id` único com `uuid4()`, propagado nas mensagens seguintes:

```
# BaseSenderAgent.SendMessageBehaviour
msg.set_metadata("conversation-id", f"conv-{self.agent.jid}-guid-{uuid.uuid4()}")
```

A arquitetura preserva este identificador em `forward_message`, mas ainda não executa correlação estrita de respostas em todos os comportamentos receptores. Assim, a solução oferece rastreabilidade robusta e base para isolamento completo em iterações futuras.

3.4 Troca de Mensagens Baseada em Objetos (Pydantic + Jsonpickle)

A troca de mensagens evita texto não estruturado e adota serialização orientada a objetos:

- jsonpickle serializa payloads Python para o corpo da mensagem ACL;
- jsonpickle descodifica no recetor antes de qualquer transformação;
- os agentes acrescentam informação contextual (ex.: `visited_agents`, `weather_temperature`) mantendo estrutura consistente;
- modelos Pydantic em `model/data_model.py` suportam evolução para validação tipada ponta-a-ponta.

Este paradigma reforça legibilidade, manutenção e qualidade semântica da comunicação distribuída.

****DIAGRAMA****

3.5 Validação de Ontologias

As ontologias suportadas são definidas em configuração central (`AgentOntologies`): `sensor_activity`, `movement_recommendation` e `matching`. Os agentes executam decisões condicionais por ontologia e performativa para encaminhar dados ao próximo componente.

No estado atual, a validação ocorre por ramos condicionais em cada behaviour; não existe ainda um bloco transversal de rejeição semântica com resposta failure padronizada para todos os casos. Esta limitação é relevante para evolução da robustez protocolar.

Exemplo simplificado do padrão usado:

```
if ontology == AgentOntologies.SENSOR_ACTIVITY:
    if performative == AgentPerformatives.REQUEST:
        await self.agent.forward_message(...)
```

3.6 Estrutura de Ficheiros do Projeto

O projeto está organizado nos seguintes módulos principais:

Table 4. Estrutura de ficheiros do projeto.

Ficheiro	Conteúdo / Responsabilidade
main.py	Arranque da API, dashboard e agentes SPADE
api/server.py	Endpoints REST, registo/heartbeat de agentes, ponte para BaseSenderAgent
agents/base_background_agent.py	Classe base com loop dedicado, register/heartbeat/unregister e forward_message
agents/base_sender_agent.py	Classe emissora de mensagens ACL a partir da API
agents/sensor_agent.py	Encaminhamento de atividade e recomendações para coordenação
agents/coordinator_agent.py	Encaminhamento central para HAR, matching e persistência
agents/har_agent.py	Etapas HAR (encaminhamento modular para EnvironmentAgent)
agents/environment_agent.py	Enriquecimento com dados ambientais (temperatura)
agents/matching_agent.py	Clustering social e geração de convites de matching
agents/database_agent.py	Persistência Firestore por ontologia
model/data_model.py	Modelos Pydantic de referência para normalização de dados
services/firebase_service.py	Operações de escrita/leitura em coleções Firebase

4 Resultados e Análise Crítica

A implementação do protótipo Dance4Life permitiu validar a viabilidade técnica de uma cadeia multiagente com integração API–SPADE–Firebase. Os testes funcionais realizados sobre os endpoints de entrada e os fluxos entre agentes confirmam que o sistema processa eventos de atividade, recomendações de movimento e matching social. Ainda assim, a análise crítica mostra um conjunto de limitações relevantes para conformidade total com os pressupostos do enunciado.

4.1 Sumário do Funcionamento e Testes

Durante as sessões de teste e execução, o comportamento emergente da rede de agentes confirmou a conformidade com as regras de negócio desenhadas em Agent UML. Verificaram-se os seguintes resultados práticos:

- Encadeamento funcional de atividade: os dados recebidos em `/collect_data_activities` percorrem `SensorAgent`, `CoordinatorAgent`, `HarAgent`, `EnvironmentAgent` e `DatabaseAgent`, culminando em persistência em `dance4life_activity`.
- Fluxo de matching operacional: pedidos de matching são processados no `ClusteringAgent`, com emissão de convites para utilizadores e persistência final em `dance4life_matching`.
- Persistência multi-coleção validada: o `DatabaseAgent` grava dados em coleções distintas (atividade, recomendação e matching), refletindo separação de responsabilidades por ontologia.
- Comunicação orientada a payload estruturado: o uso de `jsonpickle` evita parsing textual manual e mantém consistência dos atributos ao longo da cadeia.
- Monitorização de vida dos agentes: o mecanismo `register/heartbeat/unregister` no servidor API fornece visibilidade operacional sobre agentes ativos.

4.2 Análise Crítica — Estrangulamento Sequencial

Apesar dos resultados positivos, a análise crítica revela limitações estruturais que devem ser abordadas para garantir escalabilidade, resiliência e extensibilidade do sistema.

1. Conformidade FIPA-ACL parcial Embora `AGREE` e `FAILURE` estejam definidos em configuração, a implementação ativa recorre sobretudo a `REQUEST/INFORM`. O handshake semântico completo (aceitação explícita e falha protocolar) não está uniformemente aplicado.
2. Validação de ontologia não transversal A decisão por ontologia é feita em blocos condicionais por agente, sem mecanismo comum de rejeição estruturada quando uma ontologia desconhecida é recebida.
3. Encadeamento rígido e acoplamento estático Os endereços dos agentes estão hardcoded e o pipeline depende de uma ordem fixa de forwards. A ausência de descoberta dinâmica limita escalabilidade e substituição de serviços.
4. Processamento HAR ainda placeholder O `HarAgent` atua atualmente como etapa de encaminhamento. A inferência propriamente dita (modelo treinado) permanece trabalho futuro.
5. Ausência de correlação estrita de respostas O `conversation-id` é gerado e propagado, mas falta um mecanismo geral de correlação/filtragem de replies para garantir isolamento rigoroso sob concorrência elevada.
6. Fragilidades de robustez operacional Erros de processamento são maioritariamente tratados por exceções e logs locais, sem política uniforme de retries, timeouts semânticos e compensação transacional.

Em síntese, o protótipo cumpre os objetivos nucleares de arquitetura distribuída baseada em agentes e demonstra operação end-to-end. No entanto, para atingir um nível mais elevado de maturidade académica e técnica, recomenda-se reforçar a disciplina protocolar FIPA-ACL e mecanismos de resiliência.

5 Conclusões e Recomendações para Melhoria

5.1 Conclusões

O desenvolvimento do protótipo Dance4Life demonstra a viabilidade de uma infraestrutura SMA em Python/SPADE para o domínio do envelhecimento ativo, com integração funcional entre API externa, cadeia de agentes e persistência cloud. O sistema apresenta modularidade clara, isolamento de responsabilidades e capacidade de extensão por novos agentes e ontologias.

Do ponto de vista do enunciado, os resultados alcançados são positivos:

- arquitetura distribuída definida e implementada;
- classes e behaviours de agentes identificados e operacionalizados em SPADE;
- comunicação entre agentes com metadados ACL e ontologias explícitas;
- utilização de payloads estruturados com serialização jsonpickle;
- persistência de resultados em Firebase Firestore.

Em contrapartida, a conformidade com FIPA-ACL é ainda parcial no plano das performativas (ausência de handshake completo agree/failure). Esta limitação não invalida a prova de conceito, mas define prioridades objetivas para evolução.

5.2 Recomendações e Trabalho Futuro

Paralelismo de Comportamentos e Otimização Assíncrona Para reduzir latência e aumentar throughput, recomenda-se a adoção de paralelismo controlado entre ramos independentes (por exemplo, análise HAR e enriquecimento contextual quando não houver dependência direta), com coordenação por `asyncio.gather` e políticas explícitas de timeout.

Exemplo conceitual:

```
result_har, result_ctx = await asyncio.gather(
    request_har(data),
    request_context(data),
    return_exceptions=True
)
```

Conformidade Protocolar FIPA-ACL Implementar o ciclo completo request → agree → inform/failure em todos os pares comunicantes, com templates de validação por ontologia e respostas semânticas de erro consistentes.

Adição de Novos Agentes e Descoberta Dinâmica A infraestrutura atual recorre a endereços estáticos (hardcoded, e.g., `har_agent@localhost`). Em trabalhos futuros, o sistema deve implementar mecanismos de Serviço de Diretório (Directory Facilitator ou Service Facilitator), permitindo que os agentes se registrem dinamicamente e que o Coordenador descubra o serviço adequado em tempo de execução. Além disso, a arquitetura ganharia robustez com a inclusão de dois novos agentes:

- Security Agent: Para auditoria de acessos e monitorização contínua da frequência cardíaca, com autonomia para interromper sessões de dança em caso de fadiga detetada.
- Feedback Agent: Responsável por interagir de volta com o utilizador de forma proativa (e.g., recomendar uma música com base nos dados guardados), fechando o ciclo de recomendação inteligente.

Segurança de Dados e Edge Computing Considerando a sensibilidade clínica e demográfica dos dados gerados, é necessário o reforço da privacidade em conformidade com o RGPD. O desenvolvimento futuro deve englobar:

- Edge Computing: Transição do processamento cognitivo (modelos de ML emulados no HAR) para o próprio dispositivo do utilizador, garantindo que apenas parâmetros anonimizados, e não sinais biométricos brutos, viajam pela rede XMPP.
- Encriptação end-to-end: A comunicação deverá incorporar encriptação em todos os behaviours de rede.
- Aprendizagem Colaborativa Distribuída: Implementação de Federated Learning, onde os agentes partilham apenas os parâmetros do modelo, preservando a privacidade dos dados brutos.

Melhorias de Código Identificadas Na análise ao código existente, foram ainda identificadas as seguintes melhorias pontuais de implementação:

- Normalizar validação de ontologias e performativas numa função utilitária transversal em BaseBackgroundAgent.
- Introduzir correlação forte de mensagens por conversation-id e reply-to em todos os receptores.
- Evitar blocos de código repetidos de forward através de factories de behaviour parametrizáveis.
- Migrar gradualmente payloads críticos para modelos Pydantic no caminho de execução (não apenas como contrato de referência).
- Implementar testes automáticos de integração para os três fluxos principais: atividade, matching e recomendação.

O Dance4Life constitui uma prova de conceito sólida que demonstra o potencial dos Sistemas Multiagente para apoiar o envelhecimento ativo. A arquitetura é clara, modular e extensível, e a implementação em SPADE valida a capacidade dos agentes para operar de forma distribuída e assíncrona. As recomendações apresentadas fornecem um roteiro concreto para transformar o protótipo num sistema mais escalável, inteligente e alinhado com as necessidades reais da população sénior.

References

1. Bugaychenko, D., Dzuba, A.: Musical Recommendations and Personalization in a Social Network. In: Proc. RecSys '13, pp. 273–280. ACM (2013)

2. Camarinha-Matos, L.M., Castolo, O., Rosas, J.: A Multi-agent Based Platform for Virtual Communities in Elderly Care. In: IEEE ETFA (2002/2003)
3. Crista, V., Martinho, D.V., Marreiros, G.: Chat4Elderly: A Multi-Agent System for Personalized Wellness Using Generative AI and Wearable Technology. In: Proc. AAMAS 2025
4. Esmaeli, A., Ghorrati, Z., Matson, E.T.: Distributed Agent-Based Collaborative Learning in Cross-Individual Wearable Sensor-Based Human Activity Recognition. arXiv:2311.04236 (2023)
5. Ferri-Molla, I., Linares-Pellicer, J., et al.: Multi-Agent AI System for Adaptive Cognitive Training in Elderly Care. In: Proc. ICAART 2025
6. Fraile, J.A., Bajo, J., Corchado, J.M.: Multi-Agent Architecture for Dependent Environments. *Inteligencia Artificial* **13**(41) (2009)
7. Fraile Nieto, J.A., et al.: The THOMAS Architecture: A Case Study in Home Care Scenarios. Springer (2010)
8. Humayun, M., Jhanjhi, N.Z., et al.: Agent-Based Medical Health Monitoring System. *Sensors* **22**(8), 2820 (2022)
9. Ivaşcu, T., Negru, V.: Activity-Aware Vital Sign Monitoring Based on a Multi-Agent Architecture. *Sensors* **21**(12), 4181 (2021)
10. Mendes, S., Queiroz, J., Leitão, P.: Data Driven Multi-agent m-Health System to Characterize the Daily Activities of Elderly People. In: Proc. CISTI 2017
11. Özkara, M., Turan, M.: Personalized News Recommendation System. *Journal of Technology and Applied Sciences* (2022)
12. Paraschiv, E.A., Vevera, A.V., et al.: Towards Trustworthy AI Agents in Geriatric Medicine. *Future Internet* **18**, 75 (2026)
13. Sánchez San Blas, H., Mendes, A.S., et al.: A Multi-agent System for Data Fusion Techniques Applied to the IoT Enabling Physical Rehabilitation Monitoring. *Applied Sciences* **11**(1), 331 (2021)
14. Shakshuki, E., Reid, M.: Multi-Agent System Applications in Healthcare. *Procedia Computer Science* **52**, 252–261 (2015)
15. Teixeira, E., Fonseca, H., et al.: Wearable Devices for Physical Activity and Healthcare Monitoring in Elderly People. *Geriatrics* **6**(2), 38 (2021)
16. Vargemidis, D., Gerling, K., et al.: Wearable Physical Activity-Tracking Systems for Older Adults. *ACM Trans. Comput. Healthcare* **1**(1) (2020)
17. Vemuri, A., Decker, K., et al.: Multi agent architecture for automated health coaching. *Journal of Medical Systems* **45**(11), 95 (2021)
18. Ziaoddini, K.: Socially Aware Music Recommendation: A Multi-Modal Graph Neural Networks for Collaborative Music Consumption. arXiv:2511.05497 (2025)
19. Unidade Curricular de Sistemas Multiagente: ASM — Agentes (T1). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
20. Unidade Curricular de Sistemas Multiagente: ASM — Sistemas Multiagente (T2). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
21. Unidade Curricular de Sistemas Multiagente: ASM — Normas e Padrões de Comunicação (T3). Apontamentos de aula em PDF, Universidade do Minho (2025/2026)
22. Novais, P., Analide, C.: Agentes Inteligentes. Texto Pedagógico, Grupo de Inteligência Artificial, Departamento de Informática, Universidade do Minho (2006)
23. Wooldridge, M.: Intelligent Agents. In: Weiss, G. (ed.) *Multiagent Systems*, pp. 27–77. MIT Press (1999)
24. Wooldridge, M., Jennings, N.R.: Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review* **10**(2), 115–152 (1995)
25. Weiss, G. (ed.): *Multiagent Systems — A Modern Approach to Distributed Artificial Intelligence*. MIT Press (1999)

26. Durfee, E., Rosenschein, J.: Distributed Problem Solving and Multiagent Systems: Comparisons and Examples. In: Proc. International Workshop on Distributed Artificial Intelligence, Seattle (1994)

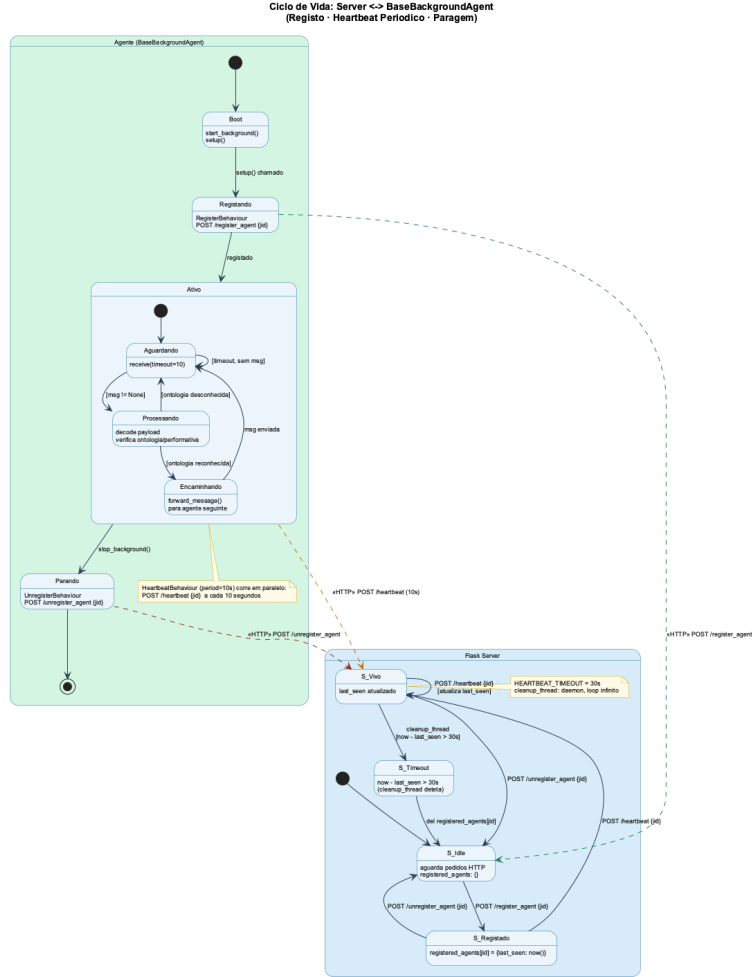


Fig. 11. Statechart — Ciclo de vida Server ↔ **BaseBackgroundAgent**: registo, heartbeat periódico (period = 10 s) e limpeza por timeout (HEARTBEAT_TIMEOUT = 30 s). As setas a tracejado representam chamadas HTTP fora do canal FIPA-ACL.

Pipeline Inter-Agentes: sensor_activity
FIPA-ACL REQUEST (→) e INFORM (--→) por salto

